

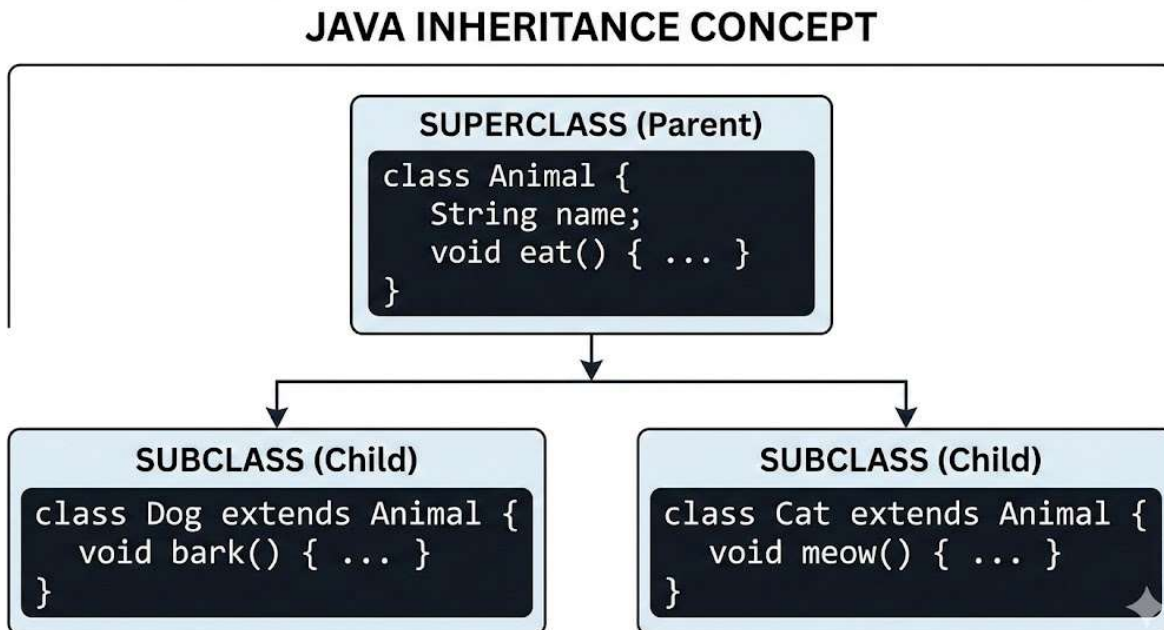
KẾ THỪA (INHERITANCE) TRONG JAVA

1. Giới thiệu chung

Kế thừa (Inheritance) là một trong bốn trụ cột chính của Lập trình hướng đối tượng (OOP), bên cạnh Đóng gói, Đa hình và Trừu tượng.

Kế thừa là cơ chế cho phép một lớp (Class) mới được xây dựng dựa trên những đặc tính và hành vi của một lớp đã tồn tại. Lớp mới sẽ sở hữu lại (kế thừa) các thuộc tính và phương thức của lớp cũ, đồng thời có thể mở rộng thêm các tính năng riêng biệt hoặc định nghĩa lại các hành vi cho phù hợp.

Mối quan hệ kế thừa đại diện cho quan hệ **"IS-A"** (Là một). Ví dụ: "Sinh viên" là một "Con người"; "Xe hơi" là một "Phương tiện".



Hình ảnh minh họa tính chất kế thừa trong Java

2. Các thuật ngữ cơ bản

Trong quan hệ kế thừa, chúng ta phân biệt hai vai trò:

- Lớp cha (Superclass / Parent Class / Base Class):** Là lớp cung cấp các thuộc tính và phương thức cho lớp khác kế thừa.
- Lớp con (Subclass / Child Class / Derived Class):** Là lớp được kế thừa từ lớp cha. Nó hưởng dụng các thành phần của lớp cha và có thể phát triển thêm các thành phần mới.

3. Cú pháp thực hiện

Trong Java, để khai báo một lớp con kế thừa từ lớp cha, ta sử dụng từ khóa `extends`.

Cú pháp:

```
class LopCha {  
    // ...  
}  
  
class LopCon extends LopCha {  
    // ...  
}
```

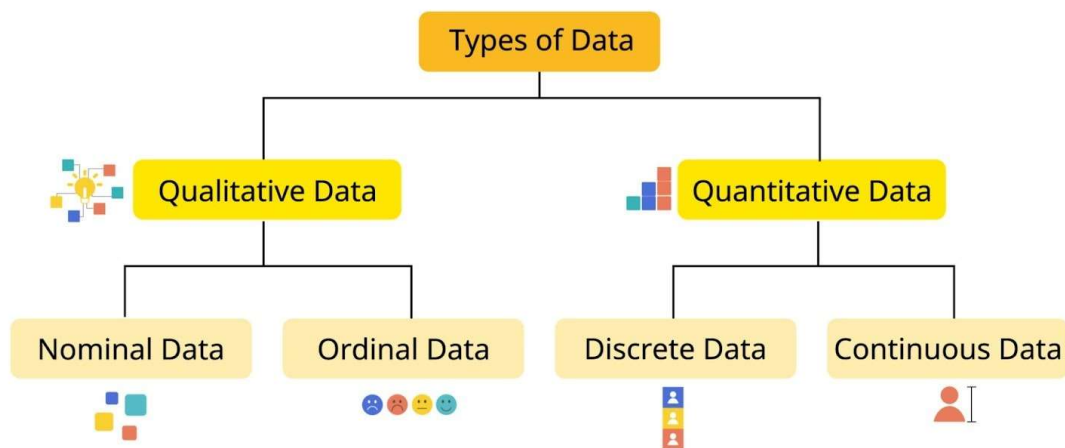
Ví dụ minh họa:

```
// Lớp cha  
class NhanVien {  
    double luong = 5000000;  
  
    void lamViec() {  
        System.out.println("Đang làm việc...");  
    }  
}  
  
// Lớp con (Lập trình viên là một Nhân viên)  
class LapTrinhVien extends NhanVien {  
    double thuong = 10000000;  
  
    // LapTrinhVien tự động có thuộc tính 'luong' và phương thức 'lamViec'  
}  
  
public class Main {  
    public static void main(String[] args) {  
        LapTrinhVien dev = new LapTrinhVien();  
  
        // Truy cập thành phần của lớp cha  
        System.out.println("Luong: " + dev.luong);  
        dev.lamViec();  
  
        // Truy cập thành phần riêng của lớp con  
        System.out.println("Thưởng: " + dev.thuong);  
    }  
}
```

4. Các loại kế thừa trong Java

Java hỗ trợ các mô hình kế thừa sau đối với Class:

1. **Đơn kế thừa (Single Inheritance):** Một lớp con chỉ kế thừa từ một lớp cha duy nhất. (A -> B).
2. **Kế thừa nhiều cấp (Multilevel Inheritance):** Một lớp con kế thừa từ một lớp cha, và lớp cha đó lại kế thừa từ một lớp khác. (A -> B -> C).
3. **Kế thừa thứ bậc (Hierarchical Inheritance):** Nhiều lớp con cùng kế thừa từ một lớp cha. (A -> B, A -> C).



Lưu ý quan trọng: Java **KHÔNG** hỗ trợ Đa kế thừa (Multiple Inheritance) đối với Class. Tức là một lớp không thể dùng từ khóa `extends` với nhiều lớp cha cùng lúc (Ví dụ: `class A extends B, C` là sai cú pháp). Điều này nhằm tránh sự nhập nhằng trong trường hợp các lớp cha có phương thức trùng tên (Vấn đề hình thoi - Diamond Problem). Tuy nhiên, Java hỗ trợ đa kế thừa thông qua **Interface**.

5. Từ khóa `super`

Khi làm việc với kế thừa, từ khóa `super` đóng vai trò quan trọng để tham chiếu đến các thành phần của lớp cha gần nhất.

- `super.tenBien`: Truy cập thuộc tính của lớp cha.
- `super.tenPhuongThuc()`: Gọi phương thức của lớp cha (thường dùng khi phương thức đó đã bị lớp con ghi đè).
- `super()`: Gọi Constructor của lớp cha.

Lưu ý: Nếu sử dụng `super()` trong Constructor của lớp con, nó bắt buộc phải là câu lệnh đầu tiên.

6. Ghi đè phương thức (Method Overriding)

Trong kế thừa, lớp con có quyền định nghĩa lại nội dung của một phương thức đã có ở lớp cha để phù hợp với đặc thù của nó. Kỹ thuật này gọi là Ghi đè phương thức.

Quy tắc ghi đè:

1. Tên phương thức, danh sách tham số và kiểu trả về phải giống hệt lớp cha.

2. Phạm vi truy cập không được hẹp hơn lớp cha (Ví dụ: Cha là `protected` thì Con phải là `protected` hoặc `public`).
3. Nên sử dụng chú thích (annotation) `@Override` để trình biên dịch hỗ trợ kiểm tra lỗi.

7. Những hạn chế của Kế thừa

Mặc dù lớp con kế thừa từ lớp cha, nhưng không phải tất cả mọi thứ đều được chuyển giao:

1. **Thành phần `private`:** Lớp con không thể truy cập trực tiếp các thuộc tính hoặc phương thức `private` của lớp cha. Phải truy cập thông qua các phương thức `public` hoặc `protected` (Getter/Setter).
2. **Constructor:** Constructor của lớp cha không được kế thừa. Tuy nhiên, nó có thể được gọi thông qua `super()`.
3. **Lớp `final`:** Nếu một lớp được khai báo là `final`, nó không thể bị kế thừa bởi bất kỳ lớp nào khác.

8. Lợi ích của Kế thừa

1. **Tái sử dụng mã nguồn (Code Reusability):** Giảm thiểu việc viết lại code, giúp chương trình gọn gàng hơn.
2. **Dễ bảo trì:** Khi sửa đổi logic ở lớp cha, các lớp con cũng tự động được cập nhật.
3. **Tính mở rộng:** Dễ dàng tạo ra các lớp mới dựa trên nền tảng cũ.
4. **Hỗ trợ Đa hình:** Kế thừa là cơ sở để thực hiện tính đa hình (Polymorphism) tại thời điểm chạy.

9. Kết luận

Kế thừa là một công cụ mạnh mẽ giúp tổ chức mã nguồn theo cấu trúc phân cấp, phản ánh đúng tư duy logic thực tế. Tuy nhiên, cần sử dụng kế thừa một cách thận trọng, tránh lạm dụng tạo ra các tầng kế thừa quá sâu gây khó khăn cho việc kiểm soát chương trình.